

底层传输方式

● 概述

本文主要讲述了递点软件传输即时通讯数据的方式，数据结构。即时通讯系统通过WebSocket协议实现。

一、数据包的数据结构

ID: (server / sessionId) + 添加时间

生成时间: add到待发送队列中的时间

发送时间: 和对方连接正常并且准备发送的时间

重试次数: 因为连接异常而回到队列中等待时间重新发送的次数

来自: server / sessionId

发送至: server / sessionId

键: 字符串, 表示消息的类型

值: 任何类型

二、服务端

2.1 和客户端建立连接

客户端主动向服务端发起建立连接的请求，服务端和客户端建立连接之后，还不能和客户端进行业务相关的通讯，需要等客户端将自己小程序的Token发送给服务端，服务端对Token进行校验，通过后，生成一个会话ID（每次新的连接都生成一个），然后将会话ID发送给客户端，客户端回复已经接收到，与客户端建立业务连接。

2.2 维护一个在线的客户端列表

2.1服务端和客户端建立业务上的连接之后，服务端将客户端的WS对象、客户端的Token（解析后的）、客户端的会话ID打包，添加到数组中去。数组中全部都是在线或者等待重连的客户端。

一个用户可能有多个设备在线，也就是有多个WS客户端对象，但是他们的Token当中存储的用户信息都是同一个用户。

2.3 客户端断开连接

客户端和服务端的连接断开（可能是客户端用户退出，也可能是网络不稳定临时断开之后又重连）之后，服务端不会立刻将客户端从在线列表中去除，而是先将客户端对象置为等待重连状态并且保留大概1s，在这1秒之内，客户端还是相当于在线，所以服务器的程序不会因为客户端不在线而执行和它离线相关的操作，如果1s之内客户端重新连接，客户端发来的Token和当前等待重连的客户端对象的Token相同，就将新的客户端连接视为当前等待重连的客户端的重连操作，直接将当前等待重连的客户端对象置为已连接，然后更新会话ID和WS对象，将新的会话ID发送给客户端。如果超出了1s，就将客户端对象从在线列表中清除，视为离线。

在客户端对象重连且未被清除的这1s内，如果服务端要向客户端发送消息，因为是等待重连状态而且不被视为离线也不会将它清除在线列表，服务端要发送给客户端的WS信息会缓存在服务端，等待重连成功后，服务端立刻发送给客户端，如果1s内未完成重连被清出在线列表，那么缓存的WS消息也会被清除，然后给缓存的WS消息的调用处回复一个WS消息发送失败的错误。

每个客户端对象中都有一个WS消息的缓存队列，客户端对象等待重连的时候，在缓存队列里等待。

2.4 向客户端发送消息

向客户端发送消息时需要三个参数：sessionId、key和value。

sessionId：会话ID，字符串，首先检查会话是否还在，也就是客户端对象是否还在在线列表当中，如果没有的话返回发送失败，会话不存在的错误。

key：键，字符串。

value：值，任意数据类型，发送给客户端的消息内容。

发送消息的步骤：

- a. 检查会话是否存在，键是否为字符串
- b. 生成消息对象（包括消息ID）
- c. 将消息对象加入到会话的消息队列当中
- d. 如果客户端为已连接状态，发送消息，给调用出处返回发送成功的消息
- e. 如果客户端为等待重连状态，缓存消息，等待重连并且发送，重连时间内客户端重连成功，给客户端发送消息，给调用处返回发送成功status的对象（{status, content}）；重连时间内客户端未重连，被清出在线列表，给调用处返回发送失败的对象（{status, content}）
- f. 如果客户端为离线状态，也就是不在线列表中，给调用处返回发送失败的消息（{status, content}）

2.5 向客户端发送请求

2.4介绍了向客户端发送消息的方式，基于这个方式，再封装形成向客户端发送请求的功能，请求和WS消息不同，WS消息是只有发送，请求是有发送，并且需要客户端立刻返回内容。请求等于两个WS消息，一个从服务端到客户端，一个从客户端到服务端。

封装之后的发送请求的API，需要三个参数：sessionId、key和value。

调用发送消息的接口时，sessionId不变，key是"_request"，value是对象{key, value, requestId}，value里面的key和value才是请求的key和value。

发送消息之后，如果消息发送不成功，就返回请求失败，并且注明是消息没有发出去的错误，如果消息发送成功，但是请求尚不返回成功，而是等待requestId相同的消息（相同的sessionId、key是"_request"、value是{key, value, requestId}）返回之后，如果requestId相同的消息是成功，那么请求的状态就是成功而且返回消息的value作为请求成功的value，如果返回消息的状态是失败，那么返回给请求的调用处的结果也是失败。如果请求的服务端到客户端的消息发出之后3s之内没得到回复，给请求的调用出的结果就是失败，错误原因就是超时。

2.6 处理客户端的请求

2.5提到了服务端向客户端发送请求的逻辑，客户端也会有向服务端发送请求的逻辑，服务端收到客户端的请求之后，需要有一个函数，专门用来处理客户端发来的请求，函数是按照客户端请求的key进行区分的。

服务端收到客户端发来的请求的**客户端到服务端消息**，根据消息的value中的key，检查服务端有没有定义这个key对应的函数，如果没有定义，就直接给客户端发送一个WS消息，status为0，content是没有此类请求，如果有定义，就执行函数，如果函数执行过程中发生错误，就捕捉错误并且返回status错误content错误原因，函数超过2s执行就返回status0content执行时间过长，函数执行成功就返回status1content成功data函数执行结果，